

2. Image Classification with Linear Classifiers

L02 — Image Classification with Linear Classifiers

흐름: Image Classification 문제 → Data-Driven Approach → K-Nearest Neighbor → Hyperparameter 설정 → Linear Classifier → Softmax Classifier → SVM Loss

1. K-Nearest Neighbor
2. Linear Classifier

1. Image Classification — 컴퓨터 비전의 핵심 문제

Image Classification

미리 정해진 label 집합이 있을 때, 입력 이미지에 적절한 label 하나를 할당하는 문제.

Semantic Gap

사람은 이미지를 고양이로 인식하지만, 컴퓨터가 직접 보는 것은 숫자로 이루어진 tensor이다.

사람이 이해하는 **의미적 대상**과 컴퓨터가 입력받는 **pixel 값** 사이에 차이가 존재한다.

Image Classification의 어려움

- **Viewpoint variation:** 카메라 위치가 바뀌면 거의 모든 pixel이 달라진다.
- **Illumination:** 같은 물체도 조명과 표면 반사에 따라 RGB 값이 달라진다.
- **Background clutter:** 물체와 배경의 색이나 무늬가 비슷하면 경계를 구분하기 어렵다.
- **Occlusion:** 물체의 일부만 보여도 올바른 class를 판단해야 한다.
- **Deformation:** 같은 물체가 자세나 형태를 바꿀 수 있다.
- **Intraclass variation:** 같은 class에 속하는 개체끼리도 모양, 색, 크기가 서로 다르다.

- **Context:** 주변 배경이 물체를 판단하는 데 영향을 줄 수 있다.

규칙을 직접 작성하기 어려운 이유

과거: Edge 탐지 → corner 탐지 → pattern 또는 feature 구성 → class 연결

한계: image의 모든 변화를 사람이 규칙으로 표현하기에는 비용이 크고, 확장성이 부족하다.

2. Machine Learning: Data-Driven Approach

Data-Driven Approach

이미지를 분류하는 규칙을 직접 작성하는 대신, 데이터로부터 classifier를 학습시킨다.

1. 이미지와 label로 구성된 dataset을 수집한다.
 2. Machine Learning 알고리즘으로 classifier를 학습한다.
 3. 학습에 사용하지 않은 새로운 이미지에서 classifier를 평가한다.
-

3. Nearest Neighbor Classifier

3.1 기본 아이디어

- **Training:** 모든 training image와 label을 그대로 저장한다.
- **Prediction:** test image와 가장 비슷한 training image를 찾고, 그 이미지의 label을 반환한다.

즉, 별도의 parameter를 학습하기보다 training data를 기억한다.

3.2 Distance Metric

두 이미지가 얼마나 비슷한지를 판단하려면 distance metric

e.g.) L1 metric

3.3 학습과 예측 비용

Training example이 N 개일 때:

- Training: $O(1)$ (데이터를 저장하기만 함)
- Prediction: $O(N)$ (test image를 모든 training image와 비교)

한계: 실제 서비스에서는 training이 다소 느려도 괜찮지만 prediction은 빨라야 한다.
Nearest Neighbor는 이 요구와 반대되는 구조이다.

3.4 1-Nearest Neighbor의 Decision Region

각 위치의 label은 가장 가까운 training point 하나로 결정된다. 따라서 다음 문제가 발생할 수 있다.

- 하나의 outlier가 주변 영역을 크게 차지할 수 있다. noise에 민감하다.
 - decision boundary가 불규칙하게 형성된다.
-

4. K-Nearest Neighbors

4.1 기본 아이디어

가장 가까운 하나의 example만 사용하는 대신, 가까운 K개의 example을 찾고 다수결로 label을 결정한다.

K 가 커지면 하나의 outlier나 noise가 prediction에 미치는 영향이 감소하고 decision boundary가 비교적 부드러워진다.

4.2 L1, L2 Distance

- L1 distance — Manhattan distance
 - 좌표축 방향의 차이를 개별적으로 합산
 - feature에 민감
- L2 distance — Euclidean distance
 - 전체 공간에서의 직선거리 측정
 - 전체적인 변화 탐지

같은 K라도 distance metric이 달라지면 가장 가까운 점과 decision boundary가 달라진다.

어떤 distance metric이 적절한지는 dataset에 따라 달라진다.

5. Hyperparameters

5.1 Hyperparameter

hyperparameter: 학습 알고리즘 자체에 대해 사용자가 정해야 하는 값

- 문제와 dataset에 따라 적절한 값이 다르므로 여러 값을 실제로 평가해야 한다.
- e.g.) KNN의 hyperparameter → K, distance metric

5.2 Hyperparameter 선택 방식

비선택

- Training set에서 가장 잘 작동하는 값 선택: 새로운 데이터에서도 잘 작동한다는 보장은 없다.
- Test set에서 가장 잘 작동하는 값 선택: test data에 맞춰진 모델이 만들어진다. 이후 test 성능은 새로운 데이터에 대한 일반화 성능을 공정하게 나타내지 못한다.

선택: Train / Validation / Test Split

Dataset을 세 부분으로 나눈다.

- Train set: 모델 학습
- Validation set: hyperparameter 선택
- Test set: 최종 성능 평가

절차:

1. Train set으로 여러 모델을 학습한다.
2. Validation set에서 K, distance metric 등을 비교한다.
3. 가장 좋은 hyperparameter를 선택한다.
4. 모든 결정이 끝난 뒤 test set을 한 번만 사용한다.

5.3 Cross-Validation

Training data를 여러 fold로 나누고, 각 fold를 한 번씩 validation set으로 사용한다.

작은 dataset에서는 유용하지만 계산 비용이 커서 deep learning에서는 자주 사용되지 않는다.

예: 5-fold cross-validation

1. Training data를 5개 fold로 분할한다.
2. 하나를 validation fold로 두고 나머지 4개로 학습한다.

3. Validation fold를 바꾸며 5번 반복한다.
 4. 각 hyperparameter의 평균 성능을 비교한다.
 5. Test set은 마지막 평가에만 사용한다.
-

5.4 Pixel Distance의 한계

이미지의 의미적 유사성을 제대로 반영하지 못한다.

원본 이미지에 변형(occluded, shifted, tinted)을 적용하면, 서로 다르게 보이거나 의미 보존 정도가 다르지만 동일한 pixel distance를 가질 수 있다.

→ raw pixel distance를 사용하는 KNN이 거의 사용되지 않는다.

6. Linear Classifier

6.1 Parametric Approach

- KNN: training data 전체를 저장하고 prediction 때마다 비교한다.
- Linear Classifier: 학습 데이터를 parameter W 와 b 로 압축한다.
출력 score 중 값이 큰 class를 classifier가 더 적합하다고 판단한다.
이후 배우는 neural network에서도 기본 구성 요소인 linear layer로 사용된다.

6.2 Viewpoint

- algebraic: 행렬 연산
- Visual Viewpoint
 W 의 각 행은 하나의 class에 대응한다. 즉, 입력 이미지와 각 class template의 유사성을 계산하는 것으로 해석할 수 있다.
각 class를 하나의 template으로만 표현하기 때문에, class 내부의 다양한 형태를 충분히 나타내기 어렵다.
- Geometric Viewpoint
각 class score = linear function이며, input space에 대해 linear decision boundary를 형성한다.
→ Linear Classifier는 공간을 직선, 평면 또는 hyperplane으로 나눈다.

6.3 Linear Classifier가 해결하기 어려운 문제 (Slide 64)

하나의 linear boundary로 해결할 수 없는 비선형 구조를 정확히 분리할 수 없다.

- 같은 class가 1·3사분면에 있고 다른 class가 2·4사분면에 있는 경우
 - 특정 반지름 범위만 하나의 class인 경우
 - 하나의 class가 서로 떨어진 여러 영역에 존재하는 경우
-

7. 좋은 W를 찾는 방법

7.1 두 가지 과제

1. **loss function**을 정의 — 현재 W가 만든 class score가 얼마나 나쁜지 수치화
2. **optimization** 방법 — Loss를 최소화하는 W를 효율적으로 찾기

7.2 Dataset Loss

전체 dataset의 loss는 개별 example loss의 평균이다.

Loss가 작을수록 현재 classifier의 prediction이 정답에 가깝다.

8. Softmax Classifier

8.1 Score → Probability 변환

raw score → class probability로 해석

변환 과정:

1. 각 score에 exponential을 적용한다.
2. 전체 합으로 나누어 normalize한다.

결과:

- 모든 probability가 0 이상
- 모든 class probability의 합이 1

Softmax 입력 전의 raw score = **logit** = **unnormalized log-probability**=

8.2 Softmax Loss (Slide 77)

정답 class에 할당된 probability가 클수록 좋은 classifier이다.

- 정답 probability가 1에 가까울수록 loss는 0에 가까워진다.
- 정답 probability가 0에 가까울수록 loss는 커진다.

이는 관측된 정답 데이터의 likelihood를 최대화하는 **Maximum Likelihood Estimation**으로 해석할 수 있다.

8.3 정답 분포와 비교

정답 label은 one-hot distribution으로 표현할 수 있다.

모델의 예측 probability를 p 라고 할 때, 예측 분포와 정답 분포의 차이를 비교한다.

정답이 one-hot distribution이면 Cross Entropy는 정답 class의 negative log-probability가 된다.

8.4 Softmax Loss의 범위와 초기값 (Slide 82–83)

가능한 최솟값과 최댓값

- 최댓값: $-\infty$
- 최솟값: 0

초기화 직후의 Loss

초기 W 가 작아서 모든 class score가 거의 같다고 가정한다.

강조점

1. Image Classification의 입력은 의미가 아니라 pixel tensor이다.
2. Data-Driven Approach는 분류 규칙을 직접 작성하지 않고 데이터에서 학습한다.
3. KNN은 학습이 빠르지만 prediction이 느리며, raw pixel distance가 의미적 유사성을 잘 나타내지 못한다.
4. Hyperparameter는 validation set으로 결정하고 test set은 마지막에 한 번만 사용한다.
5. Linear Classifier는 $(f(x, W) = Wx + b)$ 로 class score를 계산한다.
6. Linear Classifier의 각 class는 하나의 weight template과 linear decision boundary로 표현된다.
7. Loss function은 현재 (W) 가 만든 score의 품질을 하나의 값으로 나타낸다.

8. Softmax는 score를 probability로 바꾸고 정답 class의 negative log-probability를 최소화한다.
9. 초기 Softmax loss는 class가 C 개일 때 $(\log C)$ 에 가까워야 한다.
10. Softmax는 margin을 넘은 뒤에도 정답 class의 상대적 confidence가 증가하면 loss가 계속 감소한다.